Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

# Simulación y análisis de congestión en redes bajo distintos esquemas de encaminamiento

Monografía Final

Cátedra: Redes de Computadoras

Docentes: Santiago Martín Henn, Facundo Nicolás Oliva Cuneo

**Grupo NetRunners**

Bastida, Lucas Ramiro
Contessi, Rubén Andrés
Garay, Ignacio Hernán
Giraudo, Juan Pablo
Peñaloza, Gonzalo Adrián
Quinteros del Castillo, Tomás Agustín
Vasconsellos Blason, Benjamín Alejandro

## Introducción

En una red de datos el tráfico nunca es parejo: depende de cuántos usuarios hay conectados en ese momento, qué aplicaciones están usando y hasta de la hora del día. Cuando ese tráfico crece demasiado y se acerca al límite que pueden manejar los enlaces o los routers, aparece la congestión, y ahí es cuando empiezan los problemas: paquetes que tardan más de lo normal, pérdidas, descartes en los dispositivos intermedios.

Para este trabajo decidimos estudiar ese problema de forma práctica. La idea es armar distintas topologías en GNS3, generar tráfico cada vez más intenso entre los nodos y capturar todo con Wireshark para ver qué va pasando a medida que la red se satura.

Nos interesan puntualmente dos cosas. Primero, qué esquema de encaminamiento usa la red: no reacciona igual una red con rutas estáticas que una que corre RIP u OSPF cuando un camino empieza a saturarse. Segundo, qué tipo de tráfico circula, porque tampoco es lo mismo TCP, que achica su ventana de transmisión cuando detecta congestión, que un flujo UDP en tiempo real, que sigue mandando datos sin importarle lo que esté pasando en la red.

La idea es ir desarrollando estos dos ejes a lo largo del trabajo y, sobre el final, comparar los resultados entre las distintas arquitecturas que simulemos, apoyándonos en los mecanismos de control de congestión que vimos en la materia.

## Objetivos

### Objetivo general

Analizar de forma experimental cómo se comportan distintas arquitecturas de red frente a condiciones de congestión creciente, viendo qué peso tienen el esquema de encaminamiento utilizado y los protocolos de capa de aplicación en el rendimiento final de la red.

### Objetivos específicos

- Armar en GNS3 al menos dos topologías con esquemas de encaminamiento distintos, para poder compararlas entre sí.
- Generar tráfico creciente entre los nodos de cada topología hasta llegar a un punto de congestión controlada.
- Usar Wireshark para capturar ese tráfico y revisar qué indicadores de congestión aparecen: pérdida de paquetes, mayor latencia, retransmisiones.
- Ver cómo responde el tráfico de distintos protocolos de aplicación ante la misma congestión, por ejemplo HTTP sobre TCP frente a un flujo UDP en tiempo real.
- Relacionar lo que observemos en las capturas con los mecanismos de control de congestión de TCP que dimos en clase (ventana de congestión, arranque lento, retransmisión rápida).
- Comparar el rendimiento de las arquitecturas simuladas según pérdida de paquetes, latencia y estabilidad del tráfico.

## 1 Herramientas y entorno

Antes de meternos con la parte experimental nos parece importante dejar documentado con qué herramientas vamos a trabajar y por qué.

Vamos a armar las topologías en GNS3 en lugar de Packet Tracer. La diferencia clave es que GNS3 corre cada router y switch como una máquina virtual real, con interfaces de red que sí existen a nivel del sistema operativo del host; Packet Tracer, en cambio, es un simulador cerrado donde el tráfico nunca sale de su propio motor interno. Packet Tracer trae su propio analizador de tráfico (el modo Simulación), pero queda restringido al marco del simulador: no es el tráfico real de una placa de red. Como para nosotros es central poder capturar con Wireshark de verdad, GNS3 es la opción que nos permite hacerlo sin ningún truco.

Para los routers vamos a usar la imagen de MikroTik CHR (*Cloud Hosted Router*), que corre RouterOS, el mismo sistema operativo de los routers MikroTik físicos. La elegimos por dos motivos: primero, porque MikroTik es una marca barata y muy común en redes chicas en Argentina, así que lo que configuremos acá se traduce directamente a lo que configuraríamos en un equipo real (retomamos esto en la Sección 4); segundo, porque la licencia de prueba gratuita de CHR alcanza para todo lo que necesitamos, con la única limitación de que capa la velocidad de cada interfaz a 1 Mbps, lo cual en realidad nos viene bien: nos da un cuello de botella ya armado sin tener que configurar *traffic shaping* aparte (volvemos sobre esto en la Sección 3).

Wireshark se conecta directamente a los enlaces virtuales de GNS3: con el botón derecho sobre cualquier cable de la topología, la opción "Start capture" abre una instancia de Wireshark escuchando esa interfaz virtual en tiempo real, igual que si fuera una placa de red física. Esto nos permite poner la captura justo en el punto de la red que nos interesa en cada corrida (por ejemplo, a la salida del nodo congestionado) sin necesidad de ningún adaptador externo.

Para generar el tráfico de prueba vamos a usar `iperf3` desde las puntas de la topología (máquinas Linux livianas dentro de GNS3), tanto en modo TCP como UDP; la Sección 3 detalla cómo lo vamos a combinar con el cuello de botella para generar congestión controlada.

## 2 Marco teórico

Esta sección desarrolla los tres ejes que ya habíamos adelantado en el plan de trabajo: control de congestión (que es el centro de la monografía), encaminamiento y capa de aplicación. En cada uno tratamos de apoyarnos primero en lo que da Stallings, que es la bibliografía de la cátedra, y sólo recurrimos a otras fuentes cuando el libro no cubre algo con el detalle que necesitamos, como pasa con los protocolos de encaminamiento concretos y con algunos mecanismos de congestión que quedaron desactualizados desde que se escribió.

### 2.1 Control de congestión

La congestión aparece cuando, en un nodo de la red (un conmutador, un router), el número de paquetes que llega empieza a acercarse al límite de lo que ese nodo puede procesar y reenviar. Cada puerto de un nodo tiene colas de entrada y de salida, y si los paquetes llegan más rápido de lo que se pueden transmitir, esas colas empiezan a crecer. Como ninguna memoria es infinita, tarde o temprano el nodo se queda sin lugar donde guardar paquetes nuevos y tiene que empezar a descartarlos (Stallings, Sección 13.1).

Lo interesante es ver qué pasa con el rendimiento de la red a medida que crece la carga. En un esquema ideal, el rendimiento crecería linealmente con la carga ofrecida hasta llegar a la capacidad máxima de la red, y a partir de ahí se mantendría constante. En la práctica no pasa eso: como las memorias de los nodos son finitas, llega un punto en el que la red entra en una congestión moderada (el rendimiento sigue creciendo pero cada vez más despacio) y después en una congestión severa, donde el rendimiento real empieza a caer aunque la carga siga aumentando. Esto pasa principalmente porque, cuando las colas se llenan, los nodos empiezan a descartar paquetes, los orígenes los retransmiten, y esa retransmisión suma todavía más carga a una red que ya estaba saturada (Stallings, Sección 13.1). El Gráfico 1 resume esta idea comparando el caso ideal con el real.

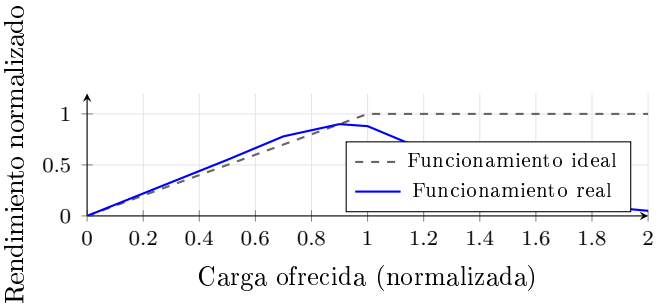



**Gráfico 1. Rendimiento ideal vs. real de una red frente al aumento de la carga ofrecida (elaboración propia en base a Stallings, Figuras 13.3 y 13.4).**

Para evitar llegar a ese escenario, Stallings describe varios mecanismos de control de congestión, cada uno pensado para intervenir en un punto distinto de la red. La contrapresión actúa a nivel de enlace: el nodo congestionado frena a sus vecinos directos, y esa restricción se va propagando hacia atrás hasta llegar al origen. Los paquetes de obstrucción son más directos: el nodo congestionado le manda al origen un paquete de control (el caso típico que da Stallings es el *Source Quench* de ICMP) pidiéndole que baje la velocidad de envío. Después están las dos formas de señalización: la implícita, que es la que usa TCP, donde el emisor infiere que hay congestión por el aumento del retardo o por la pérdida de paquetes sin que la red le avise nada explícitamente; y la explícita, donde la red marca bits en los paquetes (hacia atrás o hacia adelante, como los bits BECN/FECN en retransmisión de tramas) para notificarles a los extremos que hay congestión en el camino (Stallings, Sección 13.2).

Vale la pena una aclaración sobre estos dos últimos mecanismos, porque quedaron un poco desactualizados desde que Stallings los describió. El *Source Quench* de ICMP está formalmente deprecado desde el RFC 6633 (2013): en la práctica, casi ningún sistema operativo moderno lo genera ni lo procesa, así que si lo buscamos en nuestras capturas probablemente no lo vamos a encontrar. La señalización explícita tampoco se quedó en BECN/FECN: su equivalente moderno en redes IP es ECN (*Explicit Congestion Notification*, RFC 3168), que usa dos bits del encabezado IP para que un router marque un paquete en lugar de descartarlo cuando una cola empieza a llenarse, y el receptor le devuelve ese aviso al emisor para que reduzca la ventana sin necesidad de perder el paquete. Para que ECN tenga margen donde marcar en vez de descartar, además, conviene que los routers usen gestión activa de colas, como RED (*Random Early Detection*), en lugar del descarte de cola simple (*tail drop*) que asumimos por defecto: RED empieza a descartar o marcar paquetes al azar un poco antes de que la cola se llene del todo, para avisar la congestión de a poco en lugar de que se corten muchos flujos TCP al mismo tiempo.

Además de estos mecanismos, Stallings menciona la gestión de tráfico como un conjunto de criterios adicionales para administrar la congestión: la imparcialidad (que todos los flujos sufran la congestión en una medida parecida, en vez de que un nodo descarte siempre los paquetes del mismo origen), la calidad de servicio (dar prioridad a los flujos más sensibles al retardo, como voz o video, por sobre los que no lo son) y las reservas (acordar de antemano una velocidad de transmisión, como hacen las redes ATM) (Stallings, Sección 13.3). Esto nos va a servir más adelante para justificar por qué comparamos tráfico HTTP contra un flujo UDP en tiempo real: son dos tipos de tráfico con requisitos bien distintos frente a la congestión (Stallings, Sección 13.3).

### La reacción específica de TCP

Como una parte importante de nuestro tráfico simulado va a ser HTTP, que corre sobre TCP, nos interesa particularmente cómo reacciona TCP ante la congestión. TCP no recibe ninguna señal explícita de la red: infiere que hay congestión cuando detecta que un segmento se perdió (porque expiró un temporizador de retransmisión) o porque el retardo de ida y vuelta (RTT) empieza a crecer (Stallings, Sección 20.3).

Para controlar cuánto puede enviar en cada momento, TCP usa una ventana de congestión (*cwnd*), que se suma al mecanismo de crédito que ya conocíamos de control de flujo. Cuando se abre una conexión nueva, *cwnd* arranca en un segmento y se duplica en cada tiempo de ida y vuelta mientras lleguen confirmaciones: esto es lo que se llama arranque lento, y hace que la ventana crezca exponencialmente al principio. Cuando *cwnd* llega a un umbral (*ssthresh*), TCP pasa a una fase más conservadora, el ajuste dinámico o evitación de congestión, donde la ventana crece de a un segmento por cada tiempo de ida y vuelta en lugar de duplicarse. Si en algún momento se pierde un segmento, TCP reduce el umbral a la mitad de la ventana actual, vuelve a poner *cwnd* en uno y arranca todo el proceso de nuevo (Stallings, Sección 20.3). El Gráfico 2 ilustra cómo se ve esta evolución a lo largo del tiempo.

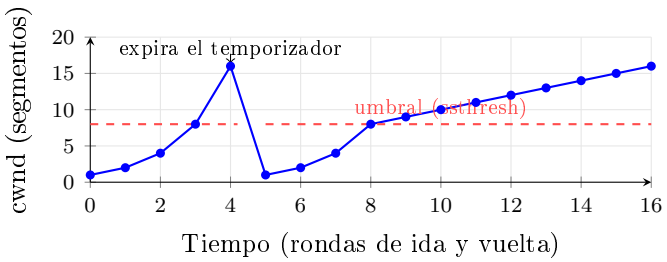



**Gráfico 2. Evolución típica de la ventana de congestión de TCP: arranque lento (crecimiento exponencial), ajuste dinámico (crecimiento lineal) y caída ante una pérdida (elaboración propia en base a Stallings, Figura 20.13).**

Lo que describimos recién es el comportamiento ante un *timeout*: TCP asume lo peor y vuelve a arrancar desde *cwnd* en uno. Pero hay otra forma de detectar la pérdida que no necesita esperar a que venza el temporizador: si el receptor sigue recibiendo segmentos fuera de orden, manda el mismo ACK repetido (ACK duplicado) por cada uno que llega; cuando el emisor ve tres ACKs duplicados seguidos, interpreta que un segmento se perdió sin que haga falta esperar el *timeout*, y lo retransmite de inmediato – esto es la retransmisión rápida. Como en este caso la red no necesariamente está tan congestionada como en un *timeout* (si siguen llegando segmentos postériores, la red todavía está entregando paquetes), TCP no vuelve a arrancar de cero: reduce el umbral a la mitad y ajusta *cwnd* a ese nuevo umbral en lugar de volver a uno, lo que se conoce como recuperación rápida. Esta diferencia es justamente uno de los indicadores que vamos a buscar en las capturas: una caída de *cwnd* por ACKs duplicados debería verse distinta – menos abrupta – que una caída por *timeout*.

Para decidir cuánto esperar antes de retransmitir un segmento, TCP necesita estimar el RTT. El algoritmo de Jacobson suaviza esa estimación combinando un promedio exponencial del RTT con una estimación de su variabilidad, en lugar de usar sólo un promedio simple, lo que lo hace mucho más sensible a cambios bruscos en las condiciones de la red (Stallings, Sección 20.3). El algoritmo de Karn, por su parte, resuelve un problema puntual: cuando un segmento se retransmite, no se puede saber con certeza si la confirmación que llegó corresponde al envío original o a la retransmisión, así que esa muestra de RTT simplemente se descarta para no distorsionar la estimación (Stallings, Sección 20.3). Estos dos mecanismos son los que vamos a buscar reflejados en las capturas de Wireshark cuando analicemos cómo reacciona TCP durante la simulación.

Una aclaración importante antes de pasar a las capturas: todo lo que describimos en esta sección (arranque lento, ajuste dinámico, caída a la mitad del umbral) corresponde a los algoritmos clásicos que desarrolla Stallings, básicamente Tahoe y Reno. El Linux que vamos a usar en las puntas de GNS3, sin embargo, usa CUBIC como algoritmo de control de congestión por defecto desde hace ya varios años, que ajusta la ventana con una función cúbica del tiempo transcurrido desde la última pérdida en lugar del crecimiento lineal de Reno, pensada para aprovechar mejor enlaces de alta velocidad y alta latencia. Lo anticipamos acá para no sorprendernos después: las capturas reales probablemente no van a calcar la forma exacta del Gráfico 2, aunque el patrón general – crecimiento, caída ante pérdida, nuevo crecimiento – se va a mantener.

### 2.2 Encaminamiento

El segundo eje tiene que ver con cómo decide la red por dónde mandar cada paquete, lo cual se vuelve clave cuando parte de la red empieza a congestionarse. Stallings presenta cuatro estrategias generales de encaminamiento, resumidas en la Tabla 1.

| Estrategia | Cómo decide la ruta | Ventaja principal | Desventaja principal |
|---|---|---|---|
| Estática | Ruta fija precalculada para cada par origen-destino | Simplicidad, buen funcionamiento con carga estable | No reacciona ante fallos ni ante congestión |
| Inundación | Cada nodo reenvía por todos los enlaces salvo por el que llegó el paquete | Máxima robustez, asegura la entrega si existe alguna ruta | Genera una cantidad de tráfico muy alta |
| Aleatoria | El nodo elige al azar (o según una probabilidad) un enlace de salida | Sencilla, reparte la carga sin necesitar información de la red | No sigue necesariamente la ruta de menor coste |
| Adaptable | Las decisiones cambian según el estado real de la red (fallos, congestión) | Mejora el rendimiento percibido y ayuda a esquivar zonas congestionadas | Más compleja y puede generar oscilaciones si reacciona mal |

**Tabla 1. Estrategias de encaminamiento (Stallings, Sección 12.2).**

Dentro del encaminamiento adaptable, que es el que nos interesa porque es el que usan los protocolos dinámicos como RIP y OSPF, Stallings desarrolla dos algoritmos de mínimo coste que son la base de casi todo el encaminamiento moderno. El algoritmo de Dijkstra necesita conocer la topología completa de la red (todos los enlaces y sus costes) y va construyendo la ruta en etapas, incorporando en cada paso el nodo más cercano que todavía no forma parte del camino; es la base de OSPF. El de Bellman-Ford es más austero: a cada nodo le alcanza con conocer el coste a sus vecinos directos, que va intercambiando periódicamente con ellos en forma de vectores de distancia; es la base de RIP (Stallings, Sección 12.3, p. 397–401).

Acá es donde encontramos el límite de Stallings para nuestro trabajo: el libro desarrolla los algoritmos de base, pero no entra en el detalle de los protocolos RIP, OSPF, EIGRP o BGP como tales. Como vamos a simular justamente RIP y OSPF en GNS3, para esta parte puntual recurrimos a Kurose y Ross, que sí los desarrolla en profundidad y es la referencia más usada en la facultad como complemento de Stallings para esto. La Tabla 2 resume la comparación que vamos a usar como base para interpretar lo que veamos en las capturas.

| | RIP | OSPF |
|---|---|---|
| Algoritmo base | Bellman-Ford (vector distancia) | Dijkstra (estado de enlace) |
| Métrica | Número de saltos (máximo 15) | Costo configurable, típicamente en función del ancho de banda |
| Temporizadores | Actualización 30 s, inválido 180 s, purga 240 s | Hello 10 s, Dead 40 s |
| Transporte | UDP puerto 520, multicast 224.0.0.9 | IP directo, protocolo número 89 |
| Convergencia | Más lenta, puede sufrir el problema de "cuenta a infinito" | Más rápida ante cambios en la red |
| Escala típica | Redes pequeñas o medianas | Redes grandes, organizadas en áreas |

**Tabla 2. RIP vs. OSPF (fuente: Kurose y Ross, *Redes de Computadoras: un Enfoque Descendente*; RFC 2453 y RFC 2328).**

Para poder interpretar lo que capturemos con Wireshark, nos interesa bajar un escalón más, de los algoritmos de base a cómo funcionan en concreto los protocolos que vamos a simular.

RIPv2 (RFC 2453) manda su tabla completa de rutas cada 30 segundos por multicast (224.0.0.9) sobre UDP, puerto 520. Si un router deja de recibir actualizaciones de un vecino durante 180 segundos, marca esa ruta como inválida, y a los 240 segundos (sin que haya llegado una actualización que la corrija) la purga definitivamente de la tabla; estos tres tiempos son justamente los que explican por qué RIP tarda tanto en darse cuenta de que algo cambió. Para evitar el problema clásico de "cuenta a infinito" (dos routers pasándose entre sí una ruta caída como si todavía existiera), RIP combina horizonte dividido (no anunciarle a un vecino una ruta que se aprendió de él mismo) con envenenamiento en reversa (anunciar esa ruta de

vuelta, pero con métrica infinita, en lugar de simplemente omitirla) y *hold-down* (ignorar por un tiempo cualquier anuncio que empeore una ruta recién caída, por si es información vieja que todavía está circulando). Las actualizaciones disparadas (mandar la tabla apenas cambia algo, sin esperar los 30 segundos) ayudan a que la convergencia no dependa únicamente del temporizador periódico.

OSPF (RFC 2328) funciona distinto: en lugar de mandar toda la tabla periódicamente, cada router arma una base de datos de estado de enlace (LSDB) a partir de anuncios (LSA, de distintos tipos según qué describan) que intercambia y sincroniza con sus vecinos solo cuando algo cambia. Los vecinos se descubren y se mantienen con paquetes Hello cada 10 segundos; si un vecino no manda un Hello en 40 segundos (el temporizador *Dead*), se lo da de baja y se recalcula. En redes multiacceso (como una LAN con varios routers), OSPF elige un router designado (DR) y uno de respaldo (BDR) para que los demás sincronicen la LSDB solo con ellos y no entre todos contra todos, y para que la red no crezca indefinidamente en una sola LSDB permite dividirla en áreas, donde el área 0 (*backbone*) conecta a todas las demás. OSPF corre directamente sobre IP, con el protocolo número 89, sin pasar por UDP ni TCP.

Dentro del mecanismo de inundación, no todos los LSA describen lo mismo. Los más relevantes para interpretar una captura son los que resume la Tabla 3.

| **Tipo** | **Nombre** | **Qué describe** |
|---|---|---|
| 1 | Router LSA | Los enlaces directos de un router y su costo; cada router lo genera únicamente para su propia área. |
| 2 | Network LSA | La red multiacceso y los routers conectados a ella; lo genera el router designado (DR) de esa red, no cada router por separado. |
| 3 | Summary LSA | Una ruta aprendida en otra área, reinundada por un router de borde de área (ABR) hacia el resto de las áreas para que sepan cómo llegar sin conocer su topología interna. |
| 5 | AS External LSA | Una ruta aprendida fuera del dominio OSPF (por ejemplo, redistribuida desde una ruta estática o desde BGP), inundada por un router de borde de sistema autónomo (ASBR). |

**Tabla 3. Tipos de LSA más comunes en OSPF (RFC 2328, Sección 12).**

En nuestra topología de laboratorio, chica y de una sola área, el tipo 1 alcanza para explicar casi todo lo que vamos a ver en las capturas: cuando cae `R2-R4`, lo que se inunda es un Router LSA actualizado de R2 y de R4, cada uno sin ese enlace en su lista. No vamos a tener tipo 2 (no hay ninguna LAN multiacceso con más de un router OSPF en nuestro diamante, todos los enlaces son punto a punto) ni tipo 3 o 5 (toda la topología vive en una única área 0, sin rutas redistribuidas desde afuera), pero nos pareció importante dejarlos documentados porque son justamente los que aparecerían en cualquier red OSPF un poco más grande que la nuestra, y porque explican por qué OSPF escala mejor que un vector-distancia plano: puede resumir y ocultar el detalle topológico de un área entera detrás de un único Summary LSA.

La métrica de OSPF merece una aclaración aparte, porque nos puede jugar en contra si no la ajustamos: el costo de un enlace se calcula como un ancho de banda de referencia dividido el ancho de banda real del enlace, y por defecto, en equipos Cisco, esa referencia es 100 Mbps. El problema es que, si no tocamos ese valor con `auto-cost reference-bandwidth`, cualquier enlace de 100 Mbps o más nos queda con costo 1, así que OSPF termina sin poder distinguir un enlace de 100 Mbps de uno de 1 Gbps y elige caminos casi como si solo contara saltos. Como en nuestras topologías vamos a tener un enlace deliberadamente angosto (el cuello de

botella que armamos a propósito, Sección 3), vamos a tener que revisar este valor para que el costo realmente refleje esa diferencia.

Para completar el panorama de las rutas estáticas que ya mencionamos en la Tabla 1: cada ruta, estática o aprendida por un protocolo, tiene asociada una distancia administrativa, que es lo que un router usa para decidir cuál preferir cuando aprende la misma ruta por más de una fuente (cuanto más baja, más confiable se la considera; una ruta estática suele tener distancia administrativa 1, más confiable que RIP 120 u OSPF 110). Esto permite armar rutas estáticas flotantes: una ruta estática con distancia administrativa deliberadamente alta, que el router ignora mientras tenga una ruta mejor (por ejemplo, aprendida por OSPF) y que solo entra en uso si esa ruta mejor desaparece – una forma de darle a la red estática algo de respaldo ante una caída sin volverla dinámica.

### Por qué vector-distancia converge más lento que estado de enlace

La Tabla 2 ya anticipa que RIP converge más lento que OSPF, pero vale la pena detenerse en el porqué, porque no tiene que ver con qué tan "rápido piensa" cada protocolo sino con qué tan completa es la información con la que arranca a pensar. Un router que corre RIP solo sabe el costo que le reportan sus vecinos directos – no tiene idea de cómo está conectado el resto de la red más allá de ellos –, así que cuando algo cambia, esa novedad tiene que viajar salto a salto: cada router espera (o dispara) una actualización, la procesa, decide si cambió su propia mejor ruta y recién ahí se la pasa al siguiente. Es una cadena de decisiones dependientes, y cada eslabón de esa cadena puede, además, estar temporalmente equivocado con información vieja que todavía no se actualizó – que es exactamente la raíz del problema de *cuenta a infinito* que mencionamos más arriba. Un router OSPF, en cambio, no interpreta nada: cuando recibe un LSA simplemente lo guarda en su base de datos de estado de enlace (LSDB) tal cual lo describe, y una vez que esa base está sincronizada con sus vecinos, calcula su propio camino de punta a punta con Dijkstra sin depender de lo que ningún otro router haya "decidido" mientras tanto. La diferencia de fondo es que RIP propaga conclusiones (mi costo a X es Y) mientras que OSPF propaga hechos (A está conectado a B con costo C), y una conclusión vieja se puede confundir con una nueva, mientras que un hecho viejo simplemente se sobrescribe con uno nuevo.

Para que esto no quede solo en la idea general, vale la pena ver con un ejemplo numérico simple cómo se arma concretamente el problema de cuenta a infinito: una cadena de cuatro routers (A–B–C–D) que originalmente llegan a una red detrás de A con costos 1, 2 y 3 respectivamente. El siguiente cuadro muestra qué pasaría, ronda a ronda, si el enlace A–B se cae y ninguna de las medidas de mitigación que ya mencionamos estuviera activa:

| Ronda | Qué sabe B (costo a la red de A) | Qué sabe C (costo a la red de A) |
|---|---|---|
| 0 (antes de la falla) | 1, directo | 2, vía B |
| 1 (A–B se cae) | Infinito – B lo detecta directo | 2 – todavía no se enteró |
| 2 | 3 – B escucha a C (2+1) y, sin horizonte dividido, le cree | 2 – sin cambios aún |
| 3 | 3 | 4 – C escucha a B (3+1) y le cree |
| 4 | 5 – B vuelve a escuchar a C (4+1) | 4 |
| ... | ... sigue subiendo de a uno por ronda ... | ... hasta llegar a 16, el "infinito" de RIP |

**Ejemplo numérico ilustrativo: cuenta a infinito sin horizonte dividido ni envenenamiento en reversa (no corresponde a una corrida real).**

Lo que hace que esta cadena ascendente no ocurra en la práctica es justamente el horizonte dividido con envenenamiento en reversa que describimos más arriba: C nunca le anuncia a B una ruta que aprendió del propio B (o se la anuncia con métrica infinita), así que B nunca tiene la oportunidad de creerle a C una ruta que, en el fondo, salió originalmente de él mismo. Pero esa protección tiene un costo en tiempo: el *hold-down* que la acompaña hace que un router ignore, durante un período prudencial, cualquier anuncio que empeore una ruta recién caída, por si todavía hay información vieja circulando – y ese período de prudencia es, en buena medida, la razón estructural de que RIP necesite varios segundos para converger incluso cuando la actualización disparada funciona bien, mientras que OSPF, al no depender de una cadena de reinterpretaciones sucesivas sino de una única inundación de hechos, no necesita ninguna protección equivalente ni el tiempo de prudencia que eso implica.

### El punto clave para nuestro tema

Acá aparece algo que nos parece central para el experimento: ni RIP ni OSPF reaccionan a la congestión. Sus métricas son fijas – RIP cuenta saltos, OSPF usa el ancho de banda nominal del enlace – y ninguna de las dos depende de cuánta carga real está circulando en ese momento. Esto significa que, si saturámos un enlace sin tirarlo abajo, ni RIP ni OSPF van a cambiar de ruta: van a seguir mandando tráfico por el mismo camino saturado, porque desde su punto de vista el enlace sigue arriba y con el mismo costo que antes. Esto cambia lo que podemos esperar de la comparación: la diferencia entre estática, RIP y OSPF que vamos a poder mostrar no es "cuál evita mejor la congestión", sino más bien cómo converge cada una ante una falla y qué tan rápido.

Para que esta comparación no se quede solo en eso, nos parece importante contrastarla con tres ideas más:

- **ECMP** (*Equal-Cost Multi-Path*): tanto RIP como OSPF pueden repartir tráfico entre varios caminos si tienen el mismo costo. Si diseñamos alguna de nuestras topologías con dos caminos equivalentes entre origen y destino, ahí sí podríamos ver una diferencia real de rendimiento contra la estática, que normalmente usa un solo camino fijo.
- **EIGRP** (protocolo propietario de Cisco, sin RFC asociado): a diferencia de RIP y OSPF, su métrica compuesta puede incluir la carga del enlace (uno de los llamados pesos K), aunque viene desactivada por defecto. Mencionarlo sirve para explicar por qué, en la práctica, casi nadie rutea según la carga: el riesgo son las oscilaciones de rutas que ya aparecen como desventaja del encaminamiento adaptable en la Tabla 1 (una ruta

se descongestiona, el tráfico vuelve a ella, se vuelve a congestionar, y así).

- **Ingeniería de tráfico** (MPLS-TE, SD-WAN): es la solución que sí se usa en redes reales para este problema – elegir caminos según la carga real, no solo según un costo fijo –, aunque queda fuera del alcance de nuestra simulación. La mencionamos solo como contexto de hacia dónde apunta la industria.

BGP, por su parte, también queda fuera del alcance directo de nuestra simulación, pero lo mencionamos porque es el protocolo que efectivamente usa Internet para el encaminamiento entre sistemas autónomos, definido en el RFC 4271; a diferencia de RIP y OSPF, que rutean dentro de una misma organización, BGP decide cómo se conectan organizaciones distintas entre sí.

### 2.3 Capa de aplicación

El tercer eje tiene que ver con qué protocolos de aplicación vamos a usar para generar el tráfico que después vamos a congestionar. Nos centramos en tres, porque se complementan bien para nuestro objetivo de comparar tráfico reactivo a la congestión contra tráfico que no lo es.

HTTP es el protocolo que vamos a usar para generar el tráfico principal de la simulación. Corre sobre TCP, y acá conviene ser precisos con la versión: en HTTP/1.0 cada solicitud abría y cerraba su propia conexión TCP, así que una página con varios recursos implicaba repetir todo el mecanismo de apertura de conexión – y con él, todo el arranque lento de TCP que desarrollamos en la Sección 2.1 – una y otra vez. HTTP/1.1, que es la versión que vamos a usar como base, soluciona esto con conexiones persistentes: una misma conexión TCP se reutiliza para varias solicitudes seguidas, lo que evita pagar el arranque lento en cada una (Stallings, Sección 22.2). Versiones más nuevas van todavía más lejos: HTTP/2 multiplexa varias solicitudes en paralelo sobre una única conexión TCP, y HTTP/3 directamente deja TCP y corre sobre QUIC, que viaja sobre UDP pero implementa su propio control de congestión a nivel de aplicación en lugar de depender del de TCP. Para nuestro trabajo esto importa porque, al usar HTTP/1.1 sobre TCP clásico, el tráfico que generemos sí va a mostrar los efectos del arranque lento y de la reducción de la ventana ante la congestión – que es justamente lo que nos interesa observar –, algo que con HTTP/3 sería mucho menos directo de interpretar.

El segundo protocolo que nos interesa desarrollar es RTP (*Real-time Transport Protocol*) sobre UDP, porque es el contraste real que nos faltaba frente a HTTP: en los objetivos planteamos comparar tráfico reactivo a la congestión contra un flujo UDP en tiempo real, y hasta ahora solo habíamos desarrollado SNMP, que no sirve para eso porque es tráfico de gestión, no de usuario. RTP es el protocolo que usan las aplicaciones de voz y video (una videollamada, streaming en vivo) para empaquetar audio y video sobre UDP, con número de secuencia y marca de tiempo para que el receptor pueda reordenar y reproducir a un ritmo constante. La diferencia clave con TCP/HTTP es que RTP no retransmite: si se pierde un paquete de audio de hace medio segundo, retransmitirlo no sirve de nada porque para cuando llegue ya pasó el momento de reproducirlo, así que el receptor simplemente lo salta y sigue. Lo que sí le importa a este tipo de tráfico es el *jitter* (la variación en el tiempo de llegada entre paquetes consecutivos), porque si varía demasiado el buffer de reproducción se queda sin datos o se desborda; y tolera cierta pérdida de paquetes, hasta un punto, sin que la calidad percibida se caiga demasiado. Esto es exactamente lo opuesto a cómo reacciona TCP, y es lo que vamos a poder contrastar generando un flujo RTP simulado – por ejemplo con `iperf3` en modo UDP a tasa fija – en paralelo con el tráfico HTTP.

SNMP, por su parte, lo traemos a colación por un motivo distinto. Es el protocolo estándar para la gestión de redes: una estación de gestión consulta o modifica variables (objetos MIB) de

los agentes instalados en los dispositivos de la red mediante mensajes `Get`, `Set` y notificaciones asíncronas (*traps*) (Stallings, Sección 22.3). A diferencia de HTTP, SNMP corre sobre UDP, así que no hereda ningún mecanismo de control de congestión: si se pierde un mensaje, simplemente se pierde. Nos interesa mencionarlo porque, en una red real, SNMP sería justamente la herramienta que se usaría para monitorear de forma automática los mismos indicadores de congestión que nosotros vamos a observar "a mano" con Wireshark (pérdida de paquetes, uso de los enlaces), lo que nos permite conectar nuestra simulación con cómo se manejaría este mismo problema en producción.

En conjunto, estos tres ejes nos dan el marco que necesitábamos para diseñar la parte experimental: sabemos qué mecanismos de congestión vamos a buscar en las capturas, qué protocolos de encaminamiento vamos a comparar y qué tipo de tráfico vamos a generar para poner a prueba todo esto.

## 3 Metodología

Esta sección describe cómo vamos a generar la congestión, qué vamos a medir y con qué herramienta, y cómo vamos a estructurar el análisis de los resultados.

### El cuello de botella

Antes de simular nada nos topamos con un problema práctico: en GNS3, por defecto, los enlaces virtuales no tienen ningún límite real de ancho de banda, así que si no configuramos nada, nunca vamos a lograr congestionar la red por más tráfico que generemos. Como ya mencionamos en la Sección 1, la licencia gratuita de MikroTik CHR limita cada interfaz a 1 Mbps, lo cual nos resuelve el problema casi de arranque: ese límite va a actuar como el cuello de botella de la topología. Donde haga falta un control más fino – por ejemplo, para que el cuello de botella quede en un punto específico de la red y no en todas las interfaces por igual – lo vamos a complementar con colas configuradas en el router o con `tc` (*traffic control*) directamente en las puntas Linux.

### Generación de tráfico y métricas

Para generar tráfico de prueba entre las puntas de la topología vamos a usar `iperf3`, que nos permite tanto tráfico TCP (para ver el comportamiento de HTTP) como UDP a tasa fija (para simular el flujo RTP en tiempo real que describimos en la Sección 2.3). La Tabla 4 resume qué herramienta usamos para medir cada métrica.

| **Métrica** | **Herramienta** |
|---|---|
| Throughput TCP | `iperf3` en modo TCP + I/O Graphs de Wireshark |
| Throughput, jitter y pérdida UDP | `iperf3` en modo UDP con tasa fija (`-b`) |
| Latencia | `ping` (RTT) |
| Arranque lento y caída de *cwnd* | Wireshark, Statistics → TCP Stream Graphs (gráfico de Stevens / *tcptrace*) |
| Tiempo de convergencia | Cortar un enlace y medir cuánto tardan en volver los `ping`s, capturando los paquetes de RIP/OSPF |

**Tabla 4. Métricas a registrar y herramienta usada para cada una.**

### De detectar a explicar

No alcanza con mostrar que hay una diferencia entre las capturas de las distintas topologías: hay que explicarla. Para que esto no quede librado a cómo le salga a cada uno, proponemos que cada resultado que presentemos siga siempre la misma estructura de tres pasos: primero, qué observamos concretamente en la captura (el número, el gráfico, el paquete puntual); segundo, qué predecía la teoría que desarrollamos en el marco teórico, con referencia explícita a la sección correspondiente; y tercero, por qué coincide o por qué no – y si no coincide, qué variable de nuestra simulación (por ejemplo, que usamos CUBIC en lugar de Reno, o que no ajustamos el costo de referencia de OSPF) puede explicar la diferencia.

### Etapas de trabajo

Así organizamos el trabajo:

- Primero armamos las tres topologías en GNS3 – idénticas entre sí, diferenciándose solo

en el esquema de encaminamiento (estática, RIP y OSPF) –, documentando cómo generamos el tráfico creciente y qué métricas anotamos en cada corrida.

- Con eso armado, corrimos las capturas con Wireshark en los mismos puntos de cada topología para que las corridas fueran realmente comparables entre sí.
- Después vino lo más laborioso: cruzar lo que vimos en las capturas con lo que desarrollamos acá, siguiendo la estructura de tres pasos que planteamos arriba.
- En paralelo, cerramos la sección de implementación real y costos (Sección 4) con las cotizaciones del mercado.
- Y cerramos con las conclusiones, volviendo a los objetivos que planteamos al principio para ver qué tanto los cumplimos, sin esconder las limitaciones de trabajar con una simulación y no con una red real.

## 4 Implementación real y costos

Esta sección le pone nombre, apellido y precio a los equipos, distinguiendo entre lo que usamos gratis para simular en la facultad y lo que costaría armar esto mismo con hardware real.

### Alcance: nuestra propia topología, no una red hipotética de empresa

En vez de inventar un escenario de red que no se corresponde con lo que realmente vamos a simular, decidimos cotizar lo que haría falta para armar nuestra propia topología de laboratorio con equipos físicos: dos routers (conectados entre sí por el enlace que usamos como cuello de botella), dos switches (uno de cada lado, para conectar las puntas) y dos computadoras o servidores livianos que hacen de cliente y servidor de `iperf3`. Es el mismo hardware para las tres variantes de encaminamiento que comparamos (estática, RIP, OSPF): en la práctica no haría falta comprarlo tres veces, alcanzaría con reconfigurar los mismos equipos entre corrida y corrida, así que cotizamos un solo juego.

### Simulación en GNS3: costo $0

Todo lo que usamos en la computadora para simular es gratuito o de código abierto: GNS3, Wireshark e iperf3/Linux no tienen costo de licencia, y la imagen de router que elegimos, MikroTik CHR, tiene una versión de prueba gratuita que alcanza para simular todas las funciones de ruteo que necesitamos (la única limitación es que capa la velocidad a 1 Mbps por interfaz, que como ya dijimos nos viene bien como cuello de botella). Esto es justamente parte de lo que nos interesa remarcar: la simulación no cuesta nada, pero no refleja el costo de montar la red de verdad.

### Equipos reales: tres niveles de calidad

Para la implementación real armamos tres opciones por rol, pensando en el triángulo calidad, tiempo y presupuesto. Los precios son los que cotizó uno de nuestros compañeros en el mercado argentino; los convertimos a pesos con el tipo de cambio oficial del 28 de octubre de 2026 ($1.560 por dólar, fuente: indicadores.ar) para que el número final tenga sentido en nuestro contexto, ya que un precio en dólares sin fecha ni tipo de cambio no dice mucho en la Argentina de hoy.

| Rol | Económico | Intermedio | Empresarial |
|---|---|---|---|
| Router de núcleo / borde | MikroTik hEX S o RB750Gr3 (USD 70–120) | Cisco Router serie 900 / Ubiquiti UniFi Gateway (USD 300–600) | Cisco ISR 1100 / Catalyst 8000 (USD 1.500–3.500+, más licencias IOS XE) |
| Switch | MikroTik CSS326 / TP-Link Smart Managed (USD 100–180) | Cisco Business 250/350 Series (USD 300–500) | Cisco Catalyst 9200/9300 (USD 1.800–4.000+) |
| Monitoreo (SNMP) | Zabbix / LibreNMS, software libre (USD 0) | Zabbix / LibreNMS (USD 0) | PRTG Network Monitor (desde USD 1.800) |

**Tabla 5. Equipos por rol y nivel de calidad, para armar nuestra propia topología de laboratorio con hardware real.**

| Nivel | Total aprox. (USD) | Total aprox. (ARS, $1.560/USD) |
|---|---|---|
| Económico | USD 200–350 | $312.000 – $546.000 |
| Intermedio | USD 700–1.200 | $1.092.000 – $1.872.000 |
| Empresarial | USD 5.000+ | $7.800.000+ |

**Tabla 6. Costo total aproximado por nivel, cotizado por el grupo en el mercado argentino (28/10/2026, dólar oficial $1.560 – fuente: indicadores.ar).**

### Que lo emulado coincida con lo que se compra

No es casualidad que hayamos elegido MikroTik CHR para emular los routers: corre RouterOS, el mismo sistema operativo que traen los routers MikroTik físicos, así que la configuración que armamos en GNS3 (rutas estáticas, RIP, OSPF) es literalmente la misma que usaríamos en el equipo real de nivel económico. Si hubiéramos elegido Cisco IOSv, en cambio, hubiéramos necesitado una licencia paga (CML) para emularlo, y los equipos Cisco reales son más caros e incluyen contratos de soporte – lo cual tiene sentido para el nivel empresarial, pero no para validar nuestra topología de laboratorio.

### Monitoreo

Ya mencionamos SNMP en la Sección 2.3 como el protocolo que se usaría en producción para monitorear automáticamente los mismos indicadores que nosotros observamos "a mano" con Wireshark. Acá también hay una decisión de costo: herramientas de monitoreo libres como Zabbix o LibreNMS no tienen costo de licencia y alcanzan perfectamente para una red de nuestro tamaño, mientras que una herramienta paga como PRTG Network Monitor (desde aproximadamente USD 1.800 de licencia) suma funcionalidades pensadas para redes mucho más grandes, que para nuestro caso serían un gasto innecesario.

### Calidad, tiempo y presupuesto

Cerramos con cómo pensamos ese equilibrio entre calidad, tiempo y presupuesto, aplicado a nuestro propio caso:

- La **ruta estática** es la opción más barata en equipos – no necesita routers con soporte de protocolos dinámicos avanzados – pero la más cara en horas de operación: cada cambio en la red hay que cargarlo a mano, y un error humano tarda en notarse.
- **OSPF** necesita personal más capacitado para configurarlo y mantenerlo bien (áreas, costos, autenticación), pero una vez armado escala mejor y converge mucho más rápido que RIP ante una caída.
- Para nuestro propio escenario de laboratorio, el nivel económico (USD 200–350) ya alcanza para validar todo lo que planteamos: los tres esquemas de encaminamiento, el cuello de botella y las capturas. El nivel intermedio (USD 700–1.200) tendría sentido recién si pensamos en una red que vaya a seguir creciendo después del laboratorio; el nivel empresarial (USD 5.000+) queda totalmente fuera de escala para lo que necesitamos nosotros, pero lo dejamos documentado porque muestra hasta dónde llega el mismo problema de encaminamiento cuando la red es mucho más grande.

## 5 Resultados y análisis

Esta sección presenta lo que efectivamente medimos en la simulación, siguiendo la estructura de tres pasos que fijamos en la metodología: qué observamos, qué predecía la teoría y por qué coincide o no. Todos los números salen directo de los JSON de `iperf3` y de las salidas de `wrk` que el equipo generó y versionó en cada corrida; donde reportamos una mediana, es la mediana de las cinco repeticiones de esa fila de la matriz de experimentos.

Como ya adelantamos, el nivel de detalle no es parejo entre esquemas: la curva de congestión UDP, el TCP y HTTP/1.1 de referencia, y la contención TCP+UDP/HTTP+UDP se corrieron solo sobre rutas estáticas. Esto no es un casillero vacío por falta de tiempo: como desarrollamos en la Sección 2.1, ni RIP ni OSPF cambian su métrica con la carga del enlace, así que el comportamiento del cuello de botella (la misma cola, el mismo límite de 500 Kbit/s) no tendría motivo teórico para ser distinto si repitiéramos estos ensayos con otro esquema de ruteo activo; por eso priorizamos la convergencia ante falla, que sí es sensible al esquema, antes que repetir una curva cuyo resultado la propia teoría ya anticipa. La convergencia ante falla, en cambio, ya tiene corridas formales completas y una mediana de indisponibilidad consolidada en los tres esquemas – estática, RIP y OSPF –. La Tabla 7 resume el estado de cada ensayo por esquema.

| Ensayo | Estática | RIP | OSPF |
|---|---|---|---|
| Curva de congestión UDP (8 tasas, 5 repeticiones) | Completo (40 corridas) | No priorizado* | No priorizado* |
| TCP y HTTP/1.1 de referencia | Completo (10 corridas) | No priorizado* | No priorizado* |
| Contención TCP+UDP y HTTP+UDP | Completo (10 corridas) | No priorizado* | No priorizado* |
| Convergencia ante falla (5 repeticiones) | Completo, mediana de indisponibilidad 27,3 s (rango 23,4–29,3 s) | Completo, mediana de indisponibilidad 14,6 s (rango 12,3–29,2 s) | Completo, mediana de indisponibilidad 2,4 s (rango 1,8–3,1 s) |

*No priorizado porque no cambia el resultado esperado: ni RIP ni OSPF alteran el comportamiento del cuello de botella (Sección 2.1), así que repetir estos ensayos consumiría tiempo de laboratorio sin una hipótesis nueva que contrastar.

**Tabla 7. Estado de las corridas experimentales por esquema de encaminamiento.**

### Curva de congestión UDP

**Qué observamos.** Hicimos cinco repeticiones de 60 segundos para cada una de ocho tasas de UDP ofrecidas (100K a 800K), siempre contra el mismo cuello de 500 Kbit/s en `R2-R4`. La Tabla 8 muestra la mediana de throughput, jitter y pérdida recibidos en cada tasa; agregamos también qué porcentaje de la tasa ofrecida efectivamente se retuvo, para que el punto de quiebre de la curva se lea de un vistazo sin tener que calcularlo.

| Tasa ofrecida | Throughput mediano | Jitter mediano | Pérdida mediana | % de la tasa retenido |
|---|---|---|---|---|
| 100 Kbit/s | 99,996 Kbit/s | 0,512 ms | 0,000 % | 100,0 % |
| 200 Kbit/s | 199,989 Kbit/s | 0,809 ms | 0,000 % | 100,0 % |
| 300 Kbit/s | 299,989 Kbit/s | 1,252 ms | 0,000 % | 100,0 % |
| 400 Kbit/s | 399,983 Kbit/s | 0,846 ms | 0,000 % | 100,0 % |
| 500 Kbit/s | 483,788 Kbit/s | 2,155 ms | 2,944 % | 96,8 % |
| 600 Kbit/s | 503,554 Kbit/s | 5,401 ms | 15,787 % | 83,9 % |
| 700 Kbit/s | 518,503 Kbit/s | 7,106 ms | 25,669 % | 74,1 % |
| 800 Kbit/s | 533,567 Kbit/s | 5,201 ms | 33,080 % | 66,7 % |

**Tabla 8. Curva de congestión UDP, esquema estático (mediana de 5 repeticiones por tasa). El color de la última columna va de verde (retención total) a rojo (mayor pérdida).**

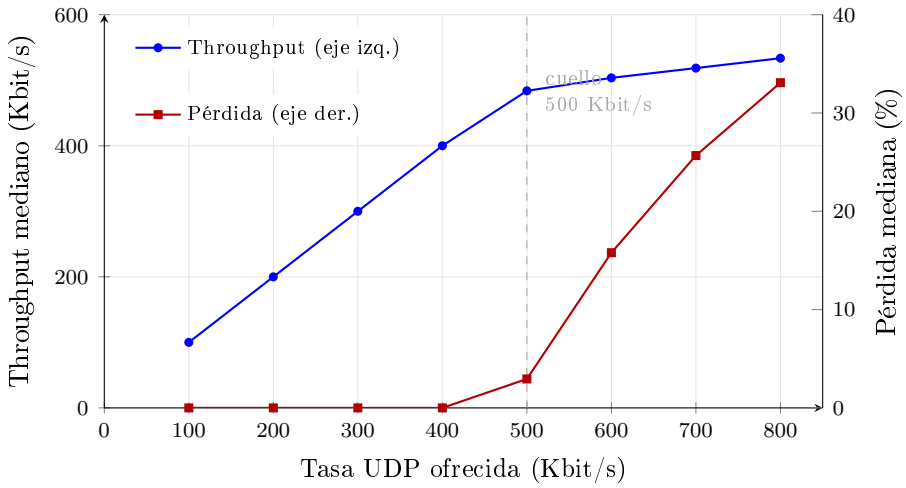



**Gráfico 3. Curva real de congestión UDP, esquema estático: throughput mediano (eje izquierdo, azul) y pérdida mediana (eje derecho, rojo) en función de la tasa ofrecida (datos de la Tabla 8).**

**Qué predecía la teoría.** En la Sección 2.1 describimos, siguiendo a Stallings, que el throughput real debería acompañar linealmente a la carga ofrecida hasta un punto, y después entrar primero en congestión moderada (el throughput sigue creciendo pero cada vez más lento) y después severa (el throughput deja de crecer o cae), tal como ilustra el Gráfico 1; el Gráfico 3 es la versión con datos reales de esa misma forma.

**Por qué coincide.** Los datos calcan la forma de esa curva casi punto por punto: hasta 400 Kbit/s el throughput sigue exactamente a la carga ofrecida y la pérdida es cero, porque la cola de R2 todavía tiene margen para absorber el tráfico sin descartar nada. A partir de 500 Kbit/s – el límite configurado en la cola `cuello-R2-R4` – el throughput deja de crecer al mismo ritmo que la carga (pasa de 483,8 a solo 533,6 Kbit/s mientras la carga ofrecida se duplica de 400K a 800K) y la pérdida sube de forma sostenida hasta el 33,1 %; en términos del % de la tasa retenido, la caída es igual de clara: del 100 % constante hasta 400 Kbit/s se pasa a apenas 66,7 % a 800 Kbit/s. A diferencia de TCP, UDP no retransmite ni reacciona a la pérdida, así que acá el mecanismo no es la avalancha de retransmisiones que describimos para TCP, sino algo más directo: la cola árbol que configuramos en R2 simplemente descarta todo paquete que exceda los 500 Kbit/s una vez que se llena (descarte de cola, *tail drop*), que es justamente el mecanismo por defecto que contrastamos en la Sección 2.1 con RED.

### TCP y HTTP/1.1 de referencia

**Qué observamos.** Para el flujo TCP de referencia (`iperf3`, sin tráfico competidor) corrimos cinco repeticiones de 60 segundos contra el mismo cuello; la Tabla 9 muestra el detalle por corrida. Para HTTP/1.1 generamos tráfico real con Nginx sirviendo un recurso de 256 KiB y `wrk` solicitándolo desde el cliente; la Tabla 10 resume las cinco corridas.

| Corrida | Throughput | Retransmisiones | RTT medio |
|---|---|---|---|
| r01 | 484,472 Kbit/s | 39 | 292,0 ms |
| r02 | 484,638 Kbit/s | 33 | 285,7 ms |
| r03 | 484,947 Kbit/s | 52 | 302,4 ms |
| r04 | 485,141 Kbit/s | 35 | 294,4 ms |
| r05 | 538,275 Kbit/s | 58 | 267,9 ms |
| **Mediana** | **484,947 Kbit/s** | **39** | **292,0 ms** |

**Tabla 9. TCP de referencia, esquema estático (`iperf3`, sin competencia).**

| Corrida | Latencia | Solicitudes/s | Transferencia |
|---|---|---|---|
| r01 | 4,37 s | 0,22 | 58,67 KB/s |
| r02 | 4,36 s | 0,22 | 58,82 KB/s |
| r03 | 4,33 s | 0,22 | 59,12 KB/s |
| r04 | 4,33 s | 0,22 | 59,12 KB/s |
| r05 | 4,34 s | 0,22 | 59,12 KB/s |
| **Mediana** | **4,34 s** | **0,22** | **59,12 KB/s** |

**Tabla 10. HTTP/1.1 de referencia, esquema estático (Nginx + `wrk`, recurso de 256 KiB).**

**Qué predecía la teoría.** Para TCP, la Sección 2.1 anticipaba que, ante un enlace angosto, iban a aparecer retransmisiones y un RTT inflado por la espera en cola, con CUBIC (no Reno/Tahoe) marcando la forma exacta de esa reacción. Para HTTP, la Sección 2.3 anticipaba que, al usar HTTP/1.1 con conexiones persistentes, el efecto dominante del cuello iba a ser el throughput disponible y no el arranque lento repetido en cada solicitud.

**Por qué coincide.** Las cinco corridas TCP efectivamente retransmitieron (mediana de 39 veces) y el RTT medio de 292 ms queda muy por encima de lo que se esperaría en una red de laboratorio sin carga – típicamente unos pocos milisegundos –, lo que es justamente el efecto de que los paquetes esperan en la cola de R2 antes de salir (lo que a veces se llama *bufferbloat*). El throughput mediano de TCP (484,9 Kbit/s) quedó apenas por debajo del límite de 500 Kbit/s de la cola, descontando el overhead de encabezados y retransmisiones. Para HTTP, la correspondencia es todavía más directa: a 59,12 KB/s (équiv. 473 Kbit/s aproximadamente) transferir un recurso de 256 KiB tarda 256 KiB $\times$ 8 / 473 Kbit/s $\approx$ 4,3 segundos, que es exactamente la latencia mediana que midió `wrk` (4,34 s) y coincide con que la conexión persistente haya logrado solo 13 solicitudes completas en el minuto de prueba. Esto confirma que, con HTTP/1.1 reutilizando la conexión, el cuello se refleja directamente en el throughput disponible, sin el ruido adicional de repetir el arranque lento en cada solicitud.

### Contención TCP+UDP y HTTP+UDP

**Qué observamos.** En estos ensayos arrancamos un flujo UDP a tasa fija (400K) y, a los 10 segundos, sumamos un segundo flujo – TCP de referencia en un caso, HTTP/1.1 en el otro – compitiendo por el mismo cuello durante el resto de la corrida. Las Tablas 11 y 12 muestran el detalle de las cinco repeticiones de cada ensayo, agregando en cada una el total combinado de los dos flujos (expresando el HTTP en Kbit/s equivalentes) para poder comparar ese total contra el cuello de 500 Kbit/s.

| Corrida | TCP | UDP | Pérdida UDP | Jitter UDP | Total |
|---|---|---|---|---|---|
| r01 | 142,2 Kbit/s | 351,2 Kbit/s | 11,76 % | 8,16 ms | 493,4 Kbit/s |
| r02 | 164,3 Kbit/s | 333,0 Kbit/s | 16,44 % | 12,36 ms | 497,3 Kbit/s |
| r03 | 150,6 Kbit/s | 344,4 Kbit/s | 13,60 % | 7,39 ms | 495,0 Kbit/s |
| r04 | 156,4 Kbit/s | 339,2 Kbit/s | 14,80 % | 7,90 ms | 495,6 Kbit/s |
| r05 | 174,7 Kbit/s | 324,3 Kbit/s | 18,60 % | 7,20 ms | 499,0 Kbit/s |
| **Mediana** | **156,4 Kbit/s** | **339,2 Kbit/s** | **14,80 %** | **7,90 ms** | **495,6 Kbit/s** |

**Tabla 11. Contención TCP+UDP, esquema estático (UDP a 400K desde los 0 s, TCP desde los 10 s). Rojo: flujo que cede ancho de banda (TCP); azul: flujo que lo retiene (UDP).**

| Corrida | Latencia HTTP | Transf. HTTP | UDP recibido | Pérdida UDP | Jitter UDP | Total equiv. |
|---|---|---|---|---|---|---|
| r01 | 13,39 s | 18,82 KB/s | 341,1 Kbit/s | 14,43 % | 6,78 ms | 491,7 Kbit/s |
| r02 | 11,53 s | 20,32 KB/s | 330,9 Kbit/s | 16,99 % | 8,30 ms | 493,5 Kbit/s |
| r03 | 13,09 s | 18,84 KB/s | 341,1 Kbit/s | 14,43 % | 6,91 ms | 491,8 Kbit/s |
| r04 | 12,16 s | 19,70 KB/s | 335,2 Kbit/s | 15,87 % | 9,08 ms | 492,8 Kbit/s |
| r05 | 11,96 s | 20,04 KB/s | 332,5 Kbit/s | 16,59 % | 8,41 ms | 492,8 Kbit/s |
| **Mediana** | **12,16 s** | **19,70 KB/s** | **335,2 Kbit/s** | **15,87 %** | **8,30 ms** | **492,8 Kbit/s** |

**Tabla 12. Contención HTTP+UDP, esquema estático (UDP a 400K en reversa desde los 0 s, HTTP desde los 10 s). Total equivalente: Kbit/s de UDP más la transferencia HTTP expresada en Kbit/s (KB/s × 8). Rojo: HTTP cede; azul: UDP retiene.**

**Qué predecía la teoría.** En la Sección 2.1 mencionamos la imparcialidad como uno de los criterios de gestión de tráfico que describe Stallings: en un esquema imparcial, todos los flujos deberían sufrir la congestión en una medida parecida. También anticipamos, en la Sección 2.3, que UDP no tiene ningún mecanismo de control de congestión – sigue mandando a tasa fija pase lo que pase –, mientras que TCP y HTTP sí reaccionan bajando su rendimiento efectivo.

**Por qué coincide.** Los dos ensayos muestran exactamente esa asimetría, y no el reparto parejo que exigiría la imparcialidad: en la contención TCP+UDP, UDP se quedó con 339,2 Kbit/s medianos (apenas por debajo de los 400 Kbit/s que ofrece) mientras TCP colapsó a 156,4 Kbit/s, menos de un tercio de los 484,9 Kbit/s que obtenía sin competencia. En HTTP+UDP pasa lo mismo: UDP mantiene 335,2 Kbit/s mientras la transferencia de HTTP cae de 59,12 a 19,70 KB/s y su latencia casi se triplica (de 4,34 a 12,16 segundos). La razón es que la cola árbol que configuramos en R2 y R4 es una cola FIFO simple, sin ningún mecanismo de reparto como WFQ: no distingue entre flujos, así que un flujo que no reacciona a la pérdida (UDP) termina quedándose con la mayor parte del ancho de banda disponible a costa del flujo que sí reacciona (TCP/HTTP). Esto es, de hecho, el mismo argumento que ya habíamos usado en la Sección 2.1 para justificar por qué nos interesaba comparar estos dos tipos de tráfico. Vale la pena notar, de todos modos, que el total combinado se mantiene muy cerca del cuello en los dos ensayos (mediana de 495,6 Kbit/s en TCP+UDP y 492,8 Kbit/s en HTTP+UDP, contra un límite de 500 Kbit/s): la cola no pierde capacidad de válvula por repartir mal entre flujos, simplemente no decide *a quién* le saca el ancho de banda.

### Convergencia ante falla

**Qué observamos.** En las cinco repeticiones de cada esquema – estática, RIP y OSPF –, deshabilitamos el enlace `R2-R4` a los 20 segundos de empezada la corrida y lo restauramos 60 segundos después, con ping continuo y tráfico HTTP corriendo en paralelo; en los tres casos `wrk` completó la corrida completa sin errores ni *timeouts*, así que el servicio nunca se cortó del todo, solo se demoró. Usando la definición que fijamos en la metodología – desde

la última respuesta antes de la falla hasta la primera de cinco respuestas consecutivas tras la conmutación –, la indisponibilidad mediana para rutas estáticas fue de 27,3 segundos (rango 23,4 a 29,3 s), para RIP de 14,6 segundos (rango 12,3 a 29,2 s) y para OSPF de 2,4 segundos (rango 1,8 a 3,1 s). Para RIP, además, confirmamos con una captura dedicada en `R5-R4` que el tráfico efectivamente cruza por el camino de respaldo durante la falla. La Tabla 13 resume la comparación.

| **Esquema** | **Cómo detecta y conmuta** | **Mediana de indisponibilidad** | **Rango** |
|---|---|---|---|
| Estática | Monitoreo activo (`check-gateway=ping`) sobre el próximo salto | 27,3 s | 23,4–29,3 s |
| RIP | Temporizadores periódicos/disparados (actualización 30 s, inválido 180 s) | 14,6 s | 12,3–29,2 s |
| OSPF | *Flooding* de LSA al caer la interfaz y recalculo de Dijkstra | 2,4 s | 1,8–3,1 s |

**Tabla 13. Indisponibilidad ante falla por esquema de encaminamiento (mediana de 5 repeticiones).**

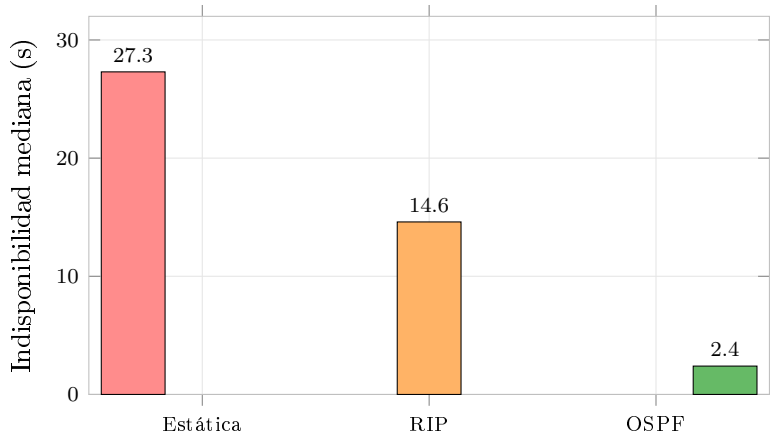



**Gráfico 4. Indisponibilidad mediana ante falla por esquema de encaminamiento (datos de la Tabla 13).**

El detalle corrida por corrida que sustenta las medianas de la Tabla 13 se muestra en las Tablas 14 y 15, para estática y RIP respectivamente: la última respuesta buena antes de la falla, el momento de la recuperación sostenida y la indisponibilidad resultante en cada una de las cinco repeticiones.

| **Corrida** | **Último ping antes de la falla** | **Reanudación sostenida** | **Indisponibilidad** |
|---|---|---|---|
| r01 | +0,291 s | +27,638 s | 27,347 s |
| r02 | +0,009 s | +23,423 s | 23,414 s |
| r03 | +0,441 s | +26,774 s | 26,334 s |
| r04 | +0,777 s | +30,086 s | 29,310 s |
| r05 | +0,848 s | +28,729 s | 27,880 s |
| **Mediana** | – | – | **27,347 s** |

**Tabla 14. Detalle de corridas, convergencia ante falla – estática (tiempos relativos al instante en que se suspende `R2-R4`).**

| Corrida | Último ping antes de la falla | Reanudación sostenida | Indisponibilidad |
|---|---|---|---|
| r01 | -0,075 s | +12,887 s | 12,963 s |
| r02 | +0,408 s | +29,591 s | 29,183 s |
| r03 | -0,008 s | +14,603 s | 14,611 s |
| r04 | +0,405 s | +26,885 s | 26,481 s |
| r05 | -0,059 s | +12,247 s | 12,306 s |
| **Mediana** | – | – | **14,611 s** |

**Tabla 15. Detalle de corridas, convergencia ante falla – RIP (tiempos relativos al instante en que se suspende `R2-R4`).**

Vale la pena notar dos cosas de esta tabla. Primero, el último ping bueno antes de la falla cae sistemáticamente un poco *después* del instante nominal de suspensión (hasta +0,85 s en algunas corridas de estática): eso no es un error de medición, es el intervalo de 0,2 s entre sondeos de `ping` sumado a la demora propia de que el script de orquestación ejecute el comando de suspensión; no afecta la indisponibilidad calculada, porque esta se mide como diferencia entre dos eventos del propio log, no contra el instante nominal del evento. Segundo, la variabilidad de RIP es visible corrida por corrida y no solo en el rango agregado: r02 y r04 tardan más del doble que r01, r03 y r05, lo que es consistente con que, según el momento exacto de la falla dentro del ciclo de 30 s, la actualización disparada le gana o no a la próxima actualización periódica.

**Qué predecía la teoría y por qué coincide.** Como desarrollamos en la Sección 2.1, ni rutas estáticas ni RIP ni OSPF reaccionan a la congestión – sus métricas no cambian con la carga –, así que la única diferencia que podíamos esperar entre esquemas no era cómo evitan saturarse, sino cómo conmutan ante una caída real del enlace; la Tabla 2 ya anticipaba que OSPF converge "más rápida ante cambios en la red" que RIP. Eso es justamente lo que confirma este ensayo, aunque con una diferencia mucho más marcada de lo que esperábamos: OSPF conmutó casi 11,5 veces más rápido que rutas estáticas (2,4 s contra 27,3 s). La razón no tiene tanto que ver con qué tan rápido converge el protocolo de ruteo en sí, sino con **cómo se entera cada esquema de que algo cambió**: las rutas estáticas dependen de que el monitoreo recursivo (`check-gateway=ping`) note, mediante sondeos periódicos propios, que el próximo salto dejó de responder, lo cual agrega una demora de detección antes de siquiera empezar a conmutar; OSPF, en cambio, detecta la caída de la interfaz casi al instante (sin depender del temporizador *Hello*/*Dead* de 10/40 s, porque acá el enlace se cae físicamente y no es un vecino que deja de saludar), inunda esa novedad mediante un LSA y recalcula el camino con Dijkstra de inmediato. RIP, con 14,6 segundos de mediana, confirma esa predicción: queda claramente entre los otros dos esquemas, más lento que OSPF pero mucho más rápido que la demora de detección de las rutas estáticas, consistente con depender de temporizadores periódicos de 30 s que las actualizaciones disparadas aceleran bastante respecto de ese peor caso.

## Conclusiones

Con la fase experimental del esquema estático completa y la convergencia ante falla medida en los tres esquemas, el trabajo deja tres conclusiones centrales, cada una ligada a un eje distinto de lo que nos propusimos estudiar: cómo se comporta la red cuando la carga supera lo que el enlace soporta, qué pasa cuando dos tipos de tráfico con comportamiento muy distinto compiten por ese mismo enlace saturado, y cuánto tarda la red en recuperarse cuando, en lugar de saturarse, directamente se corta un camino. Las tres coinciden con lo que anticipamos en el marco teórico; en alguna, como vamos a ver, coinciden pero con una magnitud que no esperábamos.

### La congestión no es un evento, es una curva – y la cruzamos exactamente donde la cola lo permite

El Gráfico 3 y la Tabla 8 muestran algo que vale la pena remarcar porque no es trivial: el quiebre de la curva no es gradual, es casi un escalón. Hasta 400 Kbit/s ofrecidos, el throughput sigue a la carga de forma perfectamente lineal y la pérdida es exactamente cero en las cinco repeticiones de cada tasa; a partir de 500 Kbit/s – el límite exacto configurado en la cola `cuello-R2-R4` – la red pasa, en un solo escalón de 100 Kbit/s de diferencia en la carga ofrecida, de retener el 100 % de lo que se le pide a retener apenas el 96,8 %, y de ahí sigue cayendo hasta el 66,7 % a 800 Kbit/s (33,1 % de pérdida). Esto es exactamente lo que predice un mecanismo de *tail drop*: mientras la cola tiene lugar, todo pasa; apenas se llena, todo lo que excede el límite se descarta sin ningún aviso previo ni degradación progresiva. No hay zona intermedia de "congestión moderada" con alguna pérdida chica y creciente antes del quiebre – hay una zona sin ninguna pérdida y después, de golpe, una zona con pérdida de dos dígitos. Esa es, de hecho, la diferencia práctica más concreta entre *tail drop* y RED que discutimos en la Sección 2.1: RED existe justamente para evitar este tipo de quiebre abrupto, descartando o marcando paquetes al azar un poco antes de que la cola se llene del todo, de manera que los flujos TCP reaccionen de a poco en lugar de que todos choquen contra la pared al mismo tiempo. Acá no tuvimos eso, y se nota.

Hay un detalle más fino en el jitter que también dice algo: crece de forma sostenida de 0,5 ms a 7,1 ms entre 100 y 700 Kbit/s, pero a 800 Kbit/s – la tasa más congestionada de todas – baja a 5,2 ms. A primera vista parecería una inconsistencia, pero tiene una explicación razonable: mientras la cola todavía tiene margen variable (encolándose a veces más, a veces menos según el instante), el tiempo de espera de cada paquete varía bastante, lo que empuja el jitter hacia arriba; una vez que la cola queda permanentemente llena – que es lo que pasa a partir de 700–800 Kbit/s –, el tiempo de encolado máximo se vuelve casi constante (la cola siempre está a tope) y lo que varía, en cambio, es cuántos paquetes se descartan directamente, no cuánto tardan los que sí pasan. Dicho de otra forma: a partir de cierto punto, la red deja de tener jitter variable y empieza a tener pérdida masiva, que es casi peor. Esto importa en la práctica porque UDP, como vimos en la Sección 2.3, no tiene ningún mecanismo propio de control de congestión: un flujo de video o voz en tiempo real que dependiera de este enlace no tendría forma de enterarse de que algo anda mal salvo por la pérdida misma, porque no hay ninguna señal intermedia – ni ECN, ni un jitter que avise antes de tiempo – que lo prepare. Toda la responsabilidad de evitar este colapso recae del lado de la red (en una cola con gestión activa) y no del emisor, precisamente porque el emisor no coopera.

### Compartir un enlace no es repartirlo: lo que separa a UDP de TCP y HTTP bajo presión

Las Tablas 11 y 12 muestran la asimetría que anticipamos en la Sección 2.1 al hablar de imparcialidad, pero los números dejan ver algo más interesante que "UDP gana y TCP pierde". Comparado con su propio desempeño sin competencia (Tabla 9: 484,9 Kbit/s), TCP cayó a 156,4 Kbit/s bajo contención – se quedó con poco menos de un tercio de lo que tenía disponible solo. HTTP tuvo el mismo destino desde otro ángulo: su transferencia cayó de 59,12 a 19,70 KB/s y su latencia casi se triplicó, de 4,34 a 12,16 segundos. UDP, en cambio, apenas se resintió: comparado con los 399,98 Kbit/s que entregaba solo a esa misma tasa de 400 Kbit/s (Tabla 8), bajo contención entregó entre 335,2 y 339,2 Kbit/s según el ensayo – una pérdida real, del orden del 15 %, pero nada parecido al colapso de TCP y HTTP.

Esa diferencia de magnitud (15 % contra más del 65 %) no se explica solo con "la cola es FIFO y no distingue flujos". Si la única causa fuera el descarte ciego de la cola, UDP y TCP deberían perder una proporción de paquetes parecida, porque ambos llegan a la misma cola y se descartan con el mismo criterio (el que llega cuando ya está llena, se pierde, sin importar el protocolo). Lo que realmente separa a los dos casos es que esa pérdida inicial – más o menos pareja – dispara reacciones completamente distintas en cada extremo: UDP la ignora y sigue mandando exactamente a la misma tasa que tenía planeada, así que su caída de throughput es, básicamente, su pérdida cruda. TCP, en cambio, interpreta esa misma pérdida como una señal de congestión (tal como describimos en la Sección 2.1: expira un temporizador de retransmisión o crece el RTT) y reacciona reduciendo a la mitad su ventana de congestión cada vez que eso pasa; con un flujo UDP constante ocupando buena parte del cuello durante toda la corrida, TCP nunca tiene margen para volver a crecer en serio antes de la próxima pérdida. El resultado es que la pérdida se multiplica dos veces: una vez en la cola (física, igual para los dos) y otra vez en el emisor TCP, que se autolimita mucho más de lo que la cola lo obligó. Es, en el fondo, el mismo mecanismo que hace que un flujo que no reacciona termine ganando frente a uno que sí reacciona: no es que la red lo favorezca, es que el que coopera paga el costo de cooperar y el que no, no.

Vale la pena notar también que el total combinado se mantuvo siempre muy cerca del límite de la cola (495,6 Kbit/s en TCP+UDP y 492,8 Kbit/s en HTTP+UDP, contra 500 Kbit/s configurados): la cola hace perfectamente su trabajo de techo, lo que no hace es repartir ese techo con ningún criterio. Esto tiene una lectura práctica directa y bastante cotidiana: es exactamente lo que le pasa a cualquier conexión hogareña chica cuando una videollamada (no reactiva, como nuestro UDP) compite con una descarga grande (reactiva, como nuestro TCP) – la videollamada sigue mandando a su tasa, la descarga se aplasta, y nadie configuró nada para que fuera así: es simplemente lo que pasa por default sin un mecanismo de reparto como WFQ o alguna forma de clasificación de tráfico (lo que Stallings llama calidad de servicio, Sección 2.1).

### El protocolo de ruteo no evita la congestión, pero decide cuánto dura apagada la red

La Tabla 13 y el Gráfico 4 muestran la comparación que, según desarrollamos en la Sección 2.1, era la única diferencia real que podíamos esperar entre esquemas de encaminamiento: no cómo evitan saturarse (ninguno lo hace), sino cómo conmutan ante una caída real del enlace. La magnitud de la diferencia, sin embargo, fue mayor a lo que anticipaba la teoría: la Tabla 2 solo decía que OSPF converge "más rápida" que RIP ante cambios en la red, sin cuantificarlo, y lo que medimos fue que OSPF conmutó casi 11,5 veces más rápido que rutas estáticas (2,4 segundos contra 27,3 segundos). Puesto en términos que cualquiera que haya usado

internet reconoce: 27 segundos sin servicio es tiempo más que suficiente para que un usuario abandone un checkout de compra o corte una videollamada; 2,4 segundos es, para la mayoría de los usos, casi imperceptible. RIP quedó, como anticipaba la teoría, en un punto intermedio: 14,6 segundos de mediana (rango 12,3 a 29,2 s), bastante más lento que OSPF pero también bastante más rápido que rutas estáticas.

Esa diferencia no viene de qué tan rápido "piensa" cada protocolo, sino de cómo se entera cada uno de que algo cambió. Las rutas estáticas no tienen ninguna noción propia de que un enlace se cayó: dependen de que el monitoreo activo (`check-gateway=ping`) note, mediante sondeos periódicos propios, que el próximo salto dejó de responder, lo que agrega una demora de detección antes incluso de empezar a conmutar. OSPF, en cambio, se entera casi al instante porque la caída es física (la interfaz se cae, no es un vecino que simplemente deja de mandar un *Hello*), inunda esa novedad mediante un LSA a toda la red y recalcula el camino con Dijkstra de inmediato – sin esperar ningún temporizador. Esto conecta directamente con algo que ya habíamos adelantado en la Sección 4: OSPF pide más conocimiento técnico para configurarlo bien (áreas, costos explícitos, autenticación) que una ruta estática, que cualquiera arma en minutos. Este resultado le pone un número concreto a esa complejidad extra: pagarla compra, en nuestra topología de laboratorio, una recuperación ante falla casi 11,5 veces más rápida. RIP, con una mediana de 14,6 segundos, confirma exactamente el punto intermedio que anticipaba la Tabla 2: no detecta la caída tan rápido como OSPF porque depende de temporizadores periódicos de 30 segundos en lugar de un aviso inmediato tipo LSA, pero tampoco queda tan expuesto como las rutas estáticas porque las actualizaciones disparadas por el propio cambio de topología aceleran bastante la propagación respecto del peor caso teórico. La variabilidad entre corridas de RIP (rango 12,3 a 29,2 segundos, mucho más amplio que el de estática u OSPF) también tiene sentido con esa misma lógica: cuánto tarda en converger depende de si la actualización disparada alcanza a viajar antes del próximo ciclo periódico de 30 segundos, algo que varía corrida a corrida según el momento exacto en que se corta el enlace.

### Relación con los objetivos

Mirando los objetivos específicos que planteamos al principio a la luz de estas tres conclusiones: armar la topología con los tres esquemas de encaminamiento quedó cumplido (los tres están configurados y validados); generar tráfico creciente hasta un punto de congestión controlada y relacionarlo con los mecanismos de TCP quedó cumplido para el esquema estático, que es el que tiene la caracterización completa; contrastar tráfico reactivo contra no reactivo quedó cubierto con un resultado más preciso del que buscábamos (no solo que UDP "gana", sino por qué gana tanto más de lo que la sola asimetría de la cola explicaría); y comparar el rendimiento de las tres arquitecturas entre sí quedó cumplido específicamente para convergencia ante falla, que terminó siendo, como anticipaba nuestro propio marco teórico, el único punto de comparación donde el esquema de ruteo realmente hace una diferencia medible.

### Limitaciones

Nos parece importante dejar explícitas las limitaciones de trabajar con una simulación y no con una red real, porque condicionan qué tan lejos se puede generalizar lo que medimos:

- La licencia gratuita de MikroTik CHR limita cada egreso a 1 Mbit/s, lo que nos obligó a fijar el cuello experimental por debajo de ese valor (500 Kbit/s) para que la saturación observada no se confundiera con el límite propio de la plataforma.
- La topología es chica – cinco routers en diamante –, pensada para que el experimento

sea manejable y no para representar una red de producción; el quiebre tan abrupto que describimos más arriba podría suavizarse en una red con más puntos de buffer intermedios.

- El tráfico TCP de `iperf3` es un flujo de referencia, no HTTP real; lo complementamos con Nginx y `wrk` para tener también tráfico HTTP/1.1 genuino, pero seguimos sin un flujo de tiempo real tipo RTP.
- La implementación de TCP de los endpoints usa CUBIC (Linux), no los algoritmos clásicos (Tahoe/Reno) que desarrollamos en el marco teórico, lo cual ya anticipamos que iba a cambiar la forma exacta de las curvas de *cwnd* – aunque no la dirección del efecto que medimos.
- Los recursos del equipo que corre la simulación acotan cuántas corridas y repeticiones pudimos hacer en paralelo.

## Bibliografía

A lo largo del marco teórico (Sección 2) citamos puntualmente cada fuente junto a la idea que respalda; esta sección reúne esas mismas fuentes en un listado único, que vamos a mantener actualizado a medida que incorporemos nuevas referencias en las secciones de Resultados y Conclusiones.


[1] Cisco Systems. *Enhanced Interior Gateway Routing Protocol (EIGRP)* – documentación técnica oficial. Disponíble en: `cisco.com`.

[2] indicadores.ar. *Cotización del dólar oficial*, consultada el 28 de octubre de 2026.

[3] Internet Engineering Task Force. *RFC 2328 – OSPF Version 2.* Disponíble en: `https://www.rfc-editor.org/rfc/rfc2328`.

[4] Internet Engineering Task Force. *RFC 2453 – RIP Version 2.* Disponíble en: `https://www.rfc-editor.org/rfc/rfc2453`.

[5] Internet Engineering Task Force. *RFC 3168 – The Addition of Explicit Congestion Notification (ECN) to IP.* Disponíble en: `https://www.rfc-editor.org/rfc/rfc3168`.

[6] Internet Engineering Task Force. *RFC 4271 – A Border Gateway Protocol 4 (BGP-4).* Disponíble en: `https://www.rfc-editor.org/rfc/rfc4271`.

[7] Internet Engineering Task Force. *RFC 6633 – Deprecation of ICMP Source Quench Messages.* Disponíble en: `https://www.rfc-editor.org/rfc/rfc6633`.

[8] Kurose, J. F. y Ross, K. W. *Redes de computadoras: un enfoque descendente.* Pearson Educación.

[9] Stallings, W. *Comunicaciones y redes de computadores.* Pearson Educación. Capítulo 12, "Encaminamiento en redes conmutadas" (págs. 379–402); Capítulo 13, "Congestión en redes de datos" (págs. 407–441); Capítulo 20, control de congestión en TCP; Capítulo 22, "Aplicaciones distribuidas" (pág. 763 en adelante).

## Anexos

Esta sección reúne el material de respaldo de la parte experimental: la topología armada en GNS3, las configuraciones de los routers y el inventario de capturas y corridas. Donde el detalle completo no entra en una tabla del informe, lo resumimos acá.

### Anexo A – Topología de red en detalle

Los tres esquemas comparten exactamente la misma topología física y el mismo direccionamiento; lo único que cambia entre ellos es la configuración de ruteo de cada router (Anexo B). El Diagrama 1 muestra la topología en diamante: el camino primario Cliente–R1–R2–R4–Servidor, con el cuello de botella de 500 Kbit/s sobre el enlace R2–R4, y el camino de respaldo R1–R3–R5–R4.

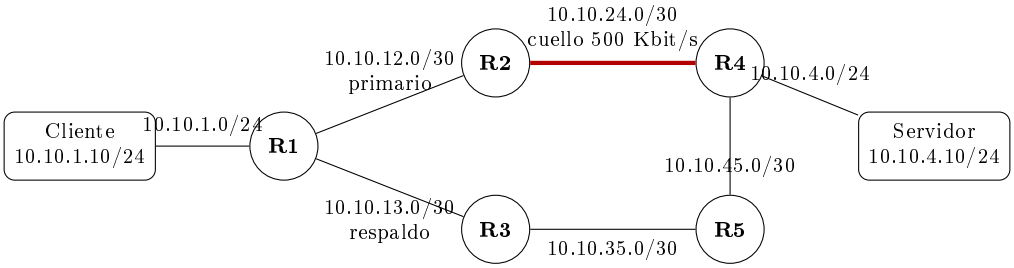
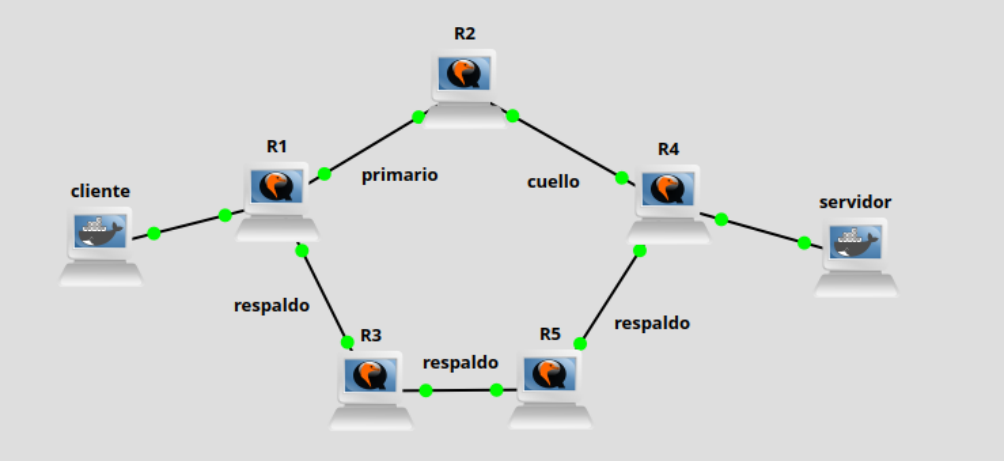


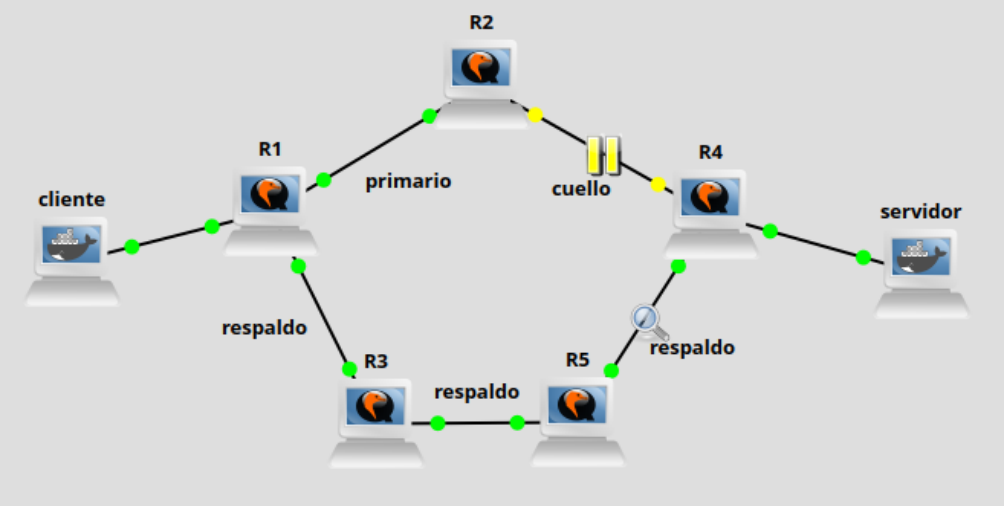

**Diagrama 1. Topología de laboratorio (diamante), idéntica en los tres esquemas de encaminamiento. La línea roja marca el enlace con el cuello de botella configurado.**

El Diagrama 1 es un esquema propio pensado para mostrar claramente el direccionamiento; la Captura 1 es la topología real tal como quedó armada en GNS3, con los cinco routers, el cliente y el servidor, y los tres enlaces de respaldo marcados en verde (enlace activo).




**Captura 1. Topología de laboratorio en GNS3 con los cinco routers MikroTik CHR, el cliente y el servidor, y todos los enlaces activos (puntos verdes).**

La Captura 2 muestra el mismo panel durante un ensayo de falla: el enlace primario R2–R4 (el del cuello de botella) queda en pausa – GNS3 lo marca con el ícono amarillo de enlace suspendido en lugar del punto verde –, mientras en el enlace de respaldo R5–R4 se ve el ícono de lupa que indica una captura de Wireshark activa, registrando el tráfico que efectivamente cruza por ese camino durante la ventana de indisponibilidad.




**Captura 2. Ensayo de falla en GNS3: el enlace R2–R4 suspendido (ícono amarillo) y una captura de Wireshark activa sobre R5–R4 (ícono de lupa), registrando el tráfico de respaldo durante el corte.**

La Tabla 16 vuelca el direccionamiento completo de cada enlace, extraído directamente de los exports de RouterOS (`/ip address` de cada router).

| Enlace | Subred | Extremo A | Extremo B |
|---|---|---|---|
| Cliente – R1 | 10.10.1.0/24 | Cliente: 10.10.1.10/24 | R1 (`ether2`): 10.10.1.1/24 |
| R1 – R2 (primario) | 10.10.12.0/30 | R1 (`ether3`): 10.10.12.1/30 | R2 (`ether2`): 10.10.12.2/30 |
| R1 – R3 (respaldo) | 10.10.13.0/30 | R1 (`ether4`): 10.10.13.1/30 | R3 (`ether2`): 10.10.13.2/30 |
| R2 – R4 (cuello) | 10.10.24.0/30 | R2 (`ether3`): 10.10.24.1/30 | R4 (`ether2`): 10.10.24.2/30 |
| R3 – R5 (respaldo) | 10.10.35.0/30 | R3 (`ether3`): 10.10.35.1/30 | R5 (`ether2`): 10.10.35.2/30 |
| R5 – R4 (respaldo) | 10.10.45.0/30 | R5 (`ether3`): 10.10.45.1/30 | R4 (`ether3`): 10.10.45.2/30 |
| R4 – Servidor | 10.10.4.0/24 | R4 (`ether4`): 10.10.4.1/24 | Servidor: 10.10.4.10/24 |

**Tabla 16. Direccionamiento IP por enlace (idéntico en estática, RIP y OSPF).**

## Anexo B – Configuraciones de routers

Cada router tiene su export de RouterOS (`export`, sin usuarios, contraseñas ni `system id`): 5 routers × 3 esquemas, 15 exports en total. El direccionamiento (Tabla 16) y la cola del cuello de botella son idénticos en los tres; lo que cambia es exclusivamente el bloque de ruteo. A continuación mostramos los fragmentos más relevantes de cada uno.

### Cola y marcado del cuello de botella (igual en los tres esquemas)

Ejemplo del export de R2 en el esquema estático; R4 tiene la regla simétrica (`cuello-R4-R2` sobre `ether2`) para limitar también el sentido servidor-cliente.

```
/queue tree
add max-limit=500k name=cuello-R2-R4 packet-mark=cuello-R2-R4 parent=ether3
/ip firewall mangle
add action=mark-packet chain=forward comment=marca-cuello-R2-R4 \
    new-packet-mark=cuello-R2-R4 out-interface=ether3 passthrough=no
```

### Rutas estáticas con monitoreo activo

Del export de R1 en el esquema estático. La ruta primaria usa `check-gateway=ping` sobre un monitoreo recursivo (no sobre R2 directamente, sino sobre la IP de R4 del otro lado del cuello); la de respaldo es una ruta flotante con distancia administrativa 2.

```
/ip route
add check-gateway=ping comment=primaria-R2 distance=1 dst-address=\
    10.10.4.0/24 gateway=10.10.24.2 target-scope=11
add comment=respaldo-R3-R5 distance=2 dst-address=10.10.4.0/24 gateway=\
    10.10.13.2
```

### RIPv2

Del export de R1 en el esquema RIP. El mismo patrón se repite en R2 a R5, aplicando la instancia a las interfaces que corresponda en cada router.

```
/routing rip instance
add name=rip-v2 redistribute=connected
/routing rip interface-template
add instance=rip-v2 interfaces=ether3,ether4
add instance=rip-v2 interfaces=ether2
```

### OSPF

Del export de R1 en el esquema OSPF. Los costos explícitos (10 para el primario, 20 para el respaldo) se repiten en R2 y R4 sobre sus interfaces correspondientes, tal como describimos en la Sección 2.1.

```
/routing ospf instance
add name=ospf-v2 redistribute=connected router-id=10.255.0.1
/routing ospf area
add area-id=0.0.0.0 instance=ospf-v2 name=backbone
/routing ospf interface-template
add area=backbone cost=10 interfaces=ether3
add area=backbone cost=20 interfaces=ether4
```

## Anexo C – Capturas de Wireshark

Todas las capturas se guardaron en formato `.pcap`. La Tabla 17 resume el inventario por esquema y ensayo; los archivos en sí no se adjuntan a este documento por su tamaño (más de 600 MB en total).

| Esquema | Ensayo | Archivos | Punto(s) de captura | Tamaño aprox. |
|---|---|---|---|---|
| Estática | Congestión (UDP 8 tasas + TCP + HTTP) | 50 | R2-R4 | 153,2 MB |
| Estática | Contención (TCP+UDP, HTTP+UDP) | 10 | R2-R4 | 37,9 MB |
| Estática | Convergencia (falla R2-R4) | 10 | R2-R4 y R5-R4 | 97,7 MB |
| RIP | Convergencia (falla R2-R4) | 10 | R2-R4 y R5-R4 | 116,6 MB |
| OSPF | Convergencia (falla R2-R4) | 10 | R2-R4 y R5-R4 | 138,9 MB |

**Tabla 17. Inventario de capturas PCAP por esquema y ensayo (90 archivos, ≈544 MB).**

Además de estas 90 capturas usadas en el análisis, se conservan otras 12 capturas de diagnóstico (≈84 MB) correspondientes a intentos descartados (por ejemplo, el primer intento de la corrida r05 de estática, interrumpido por el límite del proceso anfitrión) que no se usan para ningún número de este informe, pero se guardan como evidencia de por qué se repitió esa corrida. Como ejemplo de validación de contenido – y siguiendo la misma lógica que ya aplicamos en

la Sección 5, de no asumir ausencia de tráfico sin mirar las estadísticas completas –, la captura de `R5-R4` en la corrida r01 de RIP contiene 13.008 tramas (6.031 ICMP, 6.867 TCP y 23 RIP) y su equivalente estático contiene 395 ICMP y 5.054 TCP, confirmando en ambos casos tráfico real cruzando por el camino de respaldo durante la falla.

### Anexo D – Registro de corridas

La Tabla 18 condensa la matriz de experimentos completa (75 corridas), agrupando las cinco repeticiones de cada condición en una sola fila. Las 75 corridas fueron ejecutadas por el mismo integrante del equipo, para no mezclar variabilidad de hardware entre corridas de una misma serie; el detalle de cada repetición individual (fecha y hora exacta, parámetros literales) se registró de forma sistemática durante la ejecución.

| **Esquema** | **Ensayo** | **Tráfico / parámetro** | **Repet.** | **Fecha (UTC)** |
|---|---|---|---|---|
| Estática | Congestión | UDP, 100 Kbit/s ofrecidos | r01–r05 | 2026-11-08 |
| Estática | Congestión | UDP, 200 Kbit/s ofrecidos | r01–r05 | 2026-11-08 |
| Estática | Congestión | UDP, 300 Kbit/s ofrecidos | r01–r05 | 2026-11-08 |
| Estática | Congestión | UDP, 400 Kbit/s ofrecidos | r01–r05 | 2026-11-08 |
| Estática | Congestión | UDP, 500 Kbit/s ofrecidos | r01–r05 | 2026-11-08 |
| Estática | Congestión | UDP, 600 Kbit/s ofrecidos | r01–r05 | 2026-11-08 |
| Estática | Congestión | UDP, 700 Kbit/s ofrecidos | r01–r05 | 2026-11-08 |
| Estática | Congestión | UDP, 800 Kbit/s ofrecidos | r01–r05 | 2026-11-08 |
| Estática | Congestión | TCP (`iperf3`, referencia) | r01–r05 | 2026-11-08 |
| Estática | Congestión | HTTP/1.1 (referencia) | r01–r05 | 2026-11-08 |
| Estática | Contención | TCP+UDP (UDP 400K desde 0 s, TCP desde 10 s) | r01–r05 | 2026-11-08 |
| Estática | Contención | HTTP+UDP (UDP 400K desde 0 s, HTTP desde 10 s) | r01–r05 | 2026-11-08 |
| Estática | Convergencia | HTTP/1.1 + ICMP, falla R2-R4 a los 20 s | r01–r05 | 2026-11-08 |
| RIP | Convergencia | HTTP/1.1 + ICMP, falla R2-R4 a los 20 s | r01–r05 | 2026-11-08 |
| OSPF | Convergencia | HTTP/1.1 + ICMP, falla R2-R4 a los 20 s | r01–r05 | 2026-11-09 |

**Tabla 18. Registro condensado de corridas experimentales (15 series × 5 repeticiones = 75 corridas).**